Chapter 1
Ad Hoc Software Development

One of the first experiences a student has with a software project is the one they are given in school. A school project is not the same as an industry project. The main difference between them is the freedom you normally have in a school project. School projects have a basic initial description from the professor and an expected outcome. Often, the professor does not care how you get to the outcome. They only care that you get there. This is not true for industry projects. Industry projects have many expectations, and many of them are unexpressed yet important. One of the best ways to understand an industry project is to look at a toy example. With a toy example, you get to see the minimum things you would need to do to successfully complete a project. We will make this toy example even simpler by assuming that you are the only developer. You do not have a team. It is only you. We also assume that you were able to find a client.

Terminology:

- Developer: someone who is involved in the building of a software project. To name a few of the types of people involved in development: designer, programmer, systems, and tester. Notice that the term “developer” does not mean that they write code. They may write code, but there are many other roles to fill. In our toy example, you are the only developer, so you must perform all the roles yourself.
- Programmer: someone who codes the software project.
- Designer: someone who designs the software project on paper with UML diagrams.
- Systems: someone involved with the backend of the software project. For example: the command line, the database, the operating system, configuration files, pip, ...
- Tester: the person who tests the software project.
- Client: the person you are building the software project for.
- Product owner: there is one person in the client’s office who is principally responsible for the software project from the client’s perspective.
- Customer: your client’s customer. Technically, your client is a customer too, but we need language to separate your customer from your customer’s customers. So we say “client” for your customers, and we say “customer” for your client’s customers.
- Ad Hoc Software Development: the minimum effort to build a software project, mixing methods of project management and design on a need-by-need basis, without following a disciplined development process.

School projects and industry projects share many elements, but some minor concerns in school projects become major concerns in industry projects. One of these areas is understanding what the client wants you to create. This is true at school as well; however, the Computer Science professor usually knows what they are talking about, and the professor does not care how you got to the result (at least mostly; normally, they are looking to see that you did X inside whatever else you did that they do not care about). This makes things a little easier for you when you plan your assignment solution. Unfortunately, clients do not know Computer Science well, and they care very much about performance, ease of use, and the look and feel of the software. They also expect magic and professionalism from you. When I say magic, I mean that they think things take less time to build and at a lower price than they actually do. They also want you to implement things that may be very complex, but to them, they appear simple.

Another difference is that industry projects are often too big to build on your own, and working in a team can create many additional problems if you do not define strict development procedures, professionalism guidelines, and productivity expectations.

Toy Problem

We will use an Ad Hoc Software Development process as our initial exploration into software engineering. Ad Hoc methods are a common starting point for new software developers. It lets you see what is minimally needed and is an easy way to grasp the world of software engineering. We will start here, but we will gradually develop more mature processes as the course continues.

You are a sole developer, and your client, ABC Inc., wants you to build a digital punch card program to replace their in-house paper-based manual punch card system. You cannot believe that, in this day and age, they are still using punch cards for employees to record when they start and end work for the day. ABC Inc. pays its employees by the hour only, so this punch card system is important. They want you to bring them into the modern age, but since their employees are used to the current way of logging their work hours, they do not want you to radically change things.

That was their initial pitch to you. This is your first client, so you said yes to the project immediately. What should you do?

THE PLAN

The first thing you need to do is figure out EXACTLY what they want you to do. With the above description, I can immediately imagine a bunch of things I could implement, but which one of those ideas is the ONE they have in mind? This is your first hurdle. The client

already has the program they want you to build in their imagination. You need to discover what they have in their mind. You need to respect their wishes. You need to tell them if there is a better way that does not deviate too far from what they want.

You cannot be shy. There are no stupid questions. You must make an appointment with them to figure out exactly what they want.

A side note: the product owner is not the only person who will be using the program. The product owner might be the boss and will never use the program directly. You must meet with the product owner first, but you must also identify the program's day-to-day users. Their user experience will make or break your program's success. Be prepared to interview many people.

Another side note: do not trust your memory when interviewing people.

Further side note: each user will talk to you in their own way. Probably not in our Computer Science way of talking, and very likely from their niche industry way of thinking. I actually like this part; it requires you to put on a lot of different hats. Your sensitivity and respect will win you points.

Back to our story…

You book a 60-minute appointment with the product owner. You bring your notepad (actual paper and pencil) to the meeting with the product owner (you could bring a laptop, but there is a lot of freehand drawing that happens when you meet people, and there are no blackboards like in school, so if you bring a laptop, make sure it is touchscreen with a digital pencil). With the product owner, you want to establish the big parameters: (1) what is the big picture, (2) what are the success and failure criteria, (3) who are the users of the program (because you want to make appointments with them as well), and (4) how much will you pay me? I am going to leave out (4) in this discussion, but it will be a big part of the course later on (it is called Effort Estimation).

Clients talk to you in “stories”. In other words, non-technical language that mixes their industry language with the application they want you to build, with anecdotes.

You: “Hello, Mrs Product Owner (of course you will use their actual name – I will use PO). I would like you to describe to me what you would like me to build for you. I know we had a short conversation before, but I would like to document everything now and start going into detail. Would this be okay?”

PO: “Yes, of course. As I told you before, we need to replace our current paper punch cards with a computer. The employees currently write on the card the time they arrived, the date, and the time they left. They then give this card to the bookkeeper, who puts the cards in a

box. At the end of the week, the bookkeeper calculates the salary for all the employees and issues the checks. The punch card process works very well for us, so I do not want you to deviate too far from the current way of punching in and out of work. Everyone is used to working this way. I don't want to add confusion or long training times."

You: "What part of what you just described is not part of the application I will build?" (notice, an opposite question – what am I NOT building).

PO: "Your program will not print out the checks. We have an accounting program the bookkeeper uses to print and record all the checks. It would be good if your program prints a hardcopy report that lists all the employees, the hours they worked that week, and the total that should be written on the check. This report would be useful to both the bookkeeper and me."

You: On your paper, you immediately draw a picture of what you imagine a report of this form would look like. For actual words and numbers, you write squiggly lines, but you write actual words for the important titles. You show this immediately to the PO to get a better sense of what they actually want. You have further conversations refining the drawing.

You: "Is there anything else important about the program that I should know about?"

PO: "No."

You: "Do you have an example of the punch card you use now?"

PO: "You can get that from the bookkeeper."

You: "Can you outline for me what would make this project a success or failure?"

PO: "I can summarize both in one sentence. The bookkeeper must be able to issue checks without errors, and the bookkeeper's life should be made easier because of your application."

You: "And the employees?"

PO: "Same. No errors, and the process to record their time should take just as long or less."

You: "Do you have any requirements about the hardware?"

PO: "No. I will need you to recommend something reasonable."

You: "Do you have any requirements on how the program should look?"

PO: "Not really. I will also be looking to your recommendations about this, but I do think it should look similar to the current punch cards."

You: “Thank you for this meeting. I would like to meet the bookkeeper and at least one employee who will be using the program. I would like to get their perspectives before I give you my proposal.”

The PO gives you the information you want, and you make appointments with these people.

You arrive at the bookkeeper’s office. You introduce yourself. You tell them you met the PO and summarize what the PO said to introduce the purpose of the meeting.

You: “At my meeting with the PO, we wrote out this sample report that the program should generate. What do you think about it?”

Bookkeeper: “This would be automatically generated? You mean I would not have to handle the card anymore?”

You: “Yes, and yes.”

Bookkeeper: “This is good, but I would want to be able to search for any date range, not only the current week. Sometimes employees have issues they want to address. I will also need to be able to edit punched-in values to make corrections. Will there be a button for me to press to generate the report any time I need to?”

You: “Sure, I can do that.”

As they monologue, you record everything important in a big list on your paper. Each list is titled PO, Bookkeeper, Employee. You will want to merge and harmonize everything you heard. Sometimes what you hear is not compatible, and you will need to find common ground. In the worst case, you will go to the PO to pronounce a decision. In the very worst case, you will need to pronounce that something is impossible to do with the current technology in Computer Science (or maybe it is beyond your abilities – but that’s not a good look).

You: “Are there any additional features you would like to see?”

Bookkeeper: “It would be good if I could stay at my desk to see this report. Currently, I need to walk to the punch in/out station to collect all the cards at the end of the day.”

You: “I can make that possible.” You realize as you say this that it means there would need to be a server and a network.

You: “Does ABC Inc. already have a server and a local area network? And, are you connected to it?”

Bookkeeper: “Yes, the server is a Windows machine, and everyone’s computers are connected to the network, but there is no computer at the punch in/out station.”

You take note of this.

You: “Are there any additional requirements or issues that happen from time to time, no matter how rare, that you need to deal with?” (Clients almost never talk about rare problems, but your program will need to handle those things as well.)

Bookkeeper: “Sometimes the employees forget to record their punch-out time. For these people, I assume they left at 4 PM. Our employees normally leave at 4:30 PM. I use this as a rule to encourage them not to forget.”

You: “I could automate this punch-out for people who forget.”

Bookkeeper: “That would be great.”

You: “Anything else? What about what the program would look like?”

Bookkeeper: "Nothing else, but it would be good if the program had an easy to use menu with the keyboard keys and not the mouse. The employees' hands are often very dirty, and some of the older employees cannot use the mouse properly, for some reason.”

You make note of these things. You thank the bookkeeper for their time, and you go to see the employee. When you are in front of the employee for the first time, you give a similar introduction about why you are meeting. You give the same introduction you gave to the bookkeeper, but this time you add that you also spoke to the bookkeeper.

You: “What can you tell me about the punching in and out process, and what you would like to happen with the new program?”

Employee: “It takes us a long time to write the information on the punch card. You know, it is unpaid work, and sometimes we are in a rush at the end of the day to pick up our kids, and we don’t really have the time to write our punch-out times. Can you make some of the process automatic?”

You: “Yes.”

The employee does not tell you much more useful information, other than showing you how they fill out their punch card. They write their name, the date, the time they arrived, and the time they left. Each card has a table with seven rows. Each row has room to write the start and end time. There is also space to log which produce number they were working on. No one told you about that. This is an important nugget of information. The employee tells you that the bookkeeper picks up the cards every Friday at 5 PM.

Now you are done with your interview. You have a basic idea of what they want. A plan is forming in your mind. You will now have to make your proposal to the PO. To make things

clear, you will write out your proposal and how long you think it will take. If the PO gives the thumbs up, then you will design and build it.

PROPOSAL

Before you can start programming, you need the PO's approval and a budget. Your proposal will include the following:

- A summary of what they want
- The success and failure criteria
- The hardware and technology needed
- A basic outline of what you will build, with some drawings of what the GUI will look like and what the report will look like.
- The budget

So, what did we learn:

1. Product Owner
    a. Wants to save time and money, and reduce errors.
    b. Computation correctness is important.
    c. Wants it to look and behave similarly to the paper punch cards.
    d. Wants a report of the work hours for themselves and the bookkeeper
2. Bookkeeper
    a. Does not want to leave their desk.
    b. A Windows server and LAN exist, and everyone's computers are connected.
    c. The punch station does not have a computer.
    d. Wants to be able to edit employee times to make corrections.
    e. Wants to search any date range.
    f. The program should not use a mouse because of dirty hands and older employees.
3. Employee
    a. Automation is important.
    b. Employees also log the product number they are working on with their punch-in and punch-out times.
    c. Leaving work in a rush is a recurring problem, and the delay is caused by the punch-out process.

Our proposal:

- Add a computer at the punch station and connect it to the server.

- The computer will only have a single application installed. The punch in/out program.
- If the computer reboots, the punch in/out program automatically launches and becomes full screen.
- The program does not use a mouse. The user can use the arrow and Enter keys to make selections on the menu, or press the menu number to immediately activate the menu item.
- We will use a simple pop-up menu as the main interface: (1) Punch In, (2) Punch Out, (3) Report, (4) Quit.
- Since the report can be used by both the employees and the bookkeeper, and the bookkeeper is the only person authorized to see everyone's times, this item will have a special password. Only the bookkeeper's password gives access to all the employees' times and the edit time feature. An employee can only search their own punch-in/out times. If an employee wants to edit a time, they will need to go to the bookkeeper.
- The database will be stored on the server. It will be an SQL database.
- The punch in/out program is also stored on the server. This means it can be launched at the bookkeeper's office, so they can get the reporting they want without leaving their desk.
- Lastly, you make some basic drawings of the menus, input forms the employees will use, and the report the bookkeeper and product owner want.
- You expect it will take you 25 days to deliver. You also need to purchase an additional computer and a network connection. You are assuming the bookkeeper and product owner have printers. You will also display the report on the screen. This makes up your budget. You estimate $X.
- You give them the start date and the expected end date.

You write this up nicely and present it to the Product Owner. The PO gives you the okay.

THE DESIGN AND YOUR WORK ENVIRONMENT

Now you enter the phase where you will build the application. Before you can do that, you will need to make technical decisions. Sometimes you will talk to the client about this, but normally this is not important to them, or they do not even understand this. Often you do this by yourself. It is important that your decisions do not negatively impact the client. Your decisions should make their program even better than they expected.

In the above toy example, the following technical decisions are important:

- The Database Schema and Database Server
    - We will need a table of employees and their passwords.
    - We will need a table to store all the punched in/out records.
- Design Patterns
    - Before we start coding, we need to make the big design decisions.
- Supporting libraries
    - GUI libraries
    - Printing libraries
    - Network libraries
- IDE
    - What would be the best one to use for this case?
- Programming language(s)
    - You will need the SQL language for the database, but what about the application? We never asked if all the desktops were Windows? If all the desktops are Windows machines, we can use a compiled language like C++ for speed, but if some of the computers are Mac or Linux, we need an interpreted language like Java or Python – or even an intranet application. It does not look like the program needs to run very fast, other than at the end of the day when people want to leave quickly. There is the issue of a line-up at the punch computer at the end of the day.
- Repository (we could also talk about Pipelines, but leave that for another time)
    - Git, GitHub, GitLab?

After you have made all these decisions, you are now committed. You begin programming.

DELIVERY AND SUPPORT

The last stage of a project is delivery and support. But before we can talk about this, there are two important issues we need to talk about: Quality Assurance and Client Satisfaction. We need to make sure the program we deliver is what they want, and that it runs well at the client's building.

A rookie mistake is developing the program on your laptop at home, which is nothing like the client's computing environment, even when you try to mimic it at home.

Another rookie mistake is going home, programming the entire project, and coming to the client's building at the time you promised with your completed program, only to find out the client does not like it. There is a 100% chance that the client will not like your program. They always don't like your program. You also have the problem that it is the last day of the

project, and you have no more budget to make changes. You now need to ask for money too!

So, what can you do about it?

The solution is called Client-in-the-Loop and Incremental Installation. Instead of going away for the entire duration of the project, you meet weekly with the client to demo the program as you build it. You demo it with the product owner, the bookkeeper, and the employee. You incorporate their feedback. In addition, you install it on their server each week to make sure there are no configuration or performance issues.

At the end of the project, you do not need to install it because it is already there, and they probably already like it because they have been using it for the past month and have seen your improvements based on their feedback.

Now they like you. They pay you. They tell their friends about you.

You go home happy.